

Blog Team – Tasks for This Week

AI Coordinator – Roles & Responsibilities

AI Coordinator (@islant) will act as the bridge between AI Lead, Backend, and Frontend teams, ensuring smooth execution of AI-related tasks.

Core Responsibilities

♦ Task Coordination

- Assign AI-related tasks to team members as per AI Lead's plan
- Track progress and ensure deadlines are met

♦ Communication Bridge

- Coordinate between AI Lead, Backend Lead, and Frontend team
- Ensure everyone follows the finalized AI decisions and contracts

♦ Documentation & Updates

- Maintain and update AI documents (features, decisions, API contracts)
- Share regular status updates with the core team

♦ Quality & Alignment Check

- Ensure AI implementations follow:
 - Approved AI pipeline
 - Rule-based vs AI API decisions
- Flag wrong or inefficient approaches early

♦ Review & Reporting

- Collect review notes from AI Lead
- Report blockers, delays, or risks clearly

Day-to-Day Work

- Monitor daily AI tasks
- Sync with backend/frontend for integration readiness

- Ensure AI features move from idea → implementation → testing

Deliverables (AI Coordinator)

- ☒ Updated AI task tracker
- ☒ Clean and synced AI documentation
- ☒ Daily/weekly progress updates
- ☒ Issue & risk reports (if any)

 Just forwarding messages = Not enough

☒ Proper coordination + follow-ups = Done

AI Blog Summary Generator

Team Size: 2 Students (Swarnavi2003, vaibhav2935f)

Task Description

Build an AI-based feature that takes full blog content as input and generates a meaningful 3–5 line summary.

The summary should capture the core idea, key points, and intent of the blog in simple language.

Tech / Concepts Involved

- Text preprocessing (cleaning, tokenization)
- Keyword importance analysis
- Sentence ranking & selection
- Optional: AI APIs (OpenAI / Gemini) for better summaries

API Deliverable

Endpoint:

POST /ai/blog-summary

Input:

- Full blog text (string)

Output:

- **Short, readable summary (3–5 lines)**
-

✓ Expected Outcome

- Fast and accurate summaries
- Consistent output for repeated requests (via caching if needed)
- **Graceful fallback if AI service fails**

◆ Auto Blog Tag Generator

Assigned To: @Bhuvan23-A

🎯 Objective

Build an automated system that generates relevant tags/keywords from a blog article. These tags will be used for:

- Blog categorization
 - Better search results
 - SEO optimization
-

📝 Task Description

The system should:

1. Take blog content (text) as input
2. Analyze the text to identify important keywords
3. Return the keywords as an **array of tags**

📌 Example Output:

`["react", "nextjs", "ai", "web"]`

⚙️ Conceptual Approach

1 Text Cleaning

- Convert text to lowercase
- Remove:
 - Punctuation
 - Numbers
 - Special characters

This ensures uniform and clean data for analysis.

2 Stopword Removal

Remove commonly used words that do not add meaning as tags, such as:

is, am, are, the, and, or, this, that, etc.

This step helps focus only on meaningful keywords.

3 Keyword Extraction Techniques

You may use either or both of the following methods:

✓ Frequency-Based Keywords

- Count the occurrence of each word
- Words appearing more frequently are considered more important

✓ TF-IDF (Term Frequency – Inverse Document Frequency)

- Identifies words that are:
 - Important within the current blog
 - Not overly common across all blogs
- Helps avoid generic or overused terms

🔌 API Requirement

POST /ai/blog-tags

📁 Input

```
{  
  "content": "Your blog text here"  
}
```

Output:

```
{  
  "tags": ["react", "nextjs", "ai", "web"]  
}
```

AI Search (Semantic Search – Basic)

Assigned To: kartikrathore7773

Timeline: This Week

Objective

Build a semantic (meaning-based) blog search system that can find relevant blogs even when the exact search keywords do not match the blog content.

👉 Unlike traditional search:

- ❌ Exact keyword matching only
- ✅ Understands similar meaning and related concepts

Task Description

The system should:

1. Accept a search query from the user
2. Compare the query with all available blog content
3. Measure conceptual similarity between the query and blogs
4. Return the most relevant blogs, ranked by relevance score

Example

Search Query:

“How to build a frontend app”

Relevant Blogs Returned:

- “Getting started with React”

- “Beginner guide to web development”
-

Conceptual Approach (Suggested)

- Text preprocessing (lowercasing, cleaning)
 - Vector-based similarity (basic embeddings or TF-IDF vectors)
 - Similarity calculation (cosine similarity or equivalent)
 - Ranking blogs based on relevance score
-

API Requirement

Endpoint

POST /ai/search

Input

```
{  
  "query": "How to build a frontend app"  
}
```

Output

```
{  
  "results": [  
    {  
      "blogId": "123",  
      "title": "React Basics for Beginners",  
      "score": 0.82  
    },  
    {  
      "blogId": "456",  
      "title": "Frontend Development Guide",  
      "score": 0.65  
    }  
  ]  
}
```

✓ Expected Outcome

- Relevant blogs returned even without exact keyword match
- Results sorted by relevance score
- Fast and consistent response for user queries

Content Recommendation System

Team Size: **vrajsolanki2005**

Objective

Build a content recommendation system for the “You may also like” section that suggests relevant blogs to users based on similarity and category.

Task Description

The system should:

1. Analyze the current blog being viewed
2. Find other blogs that are similar in content or category
3. Recommend a list of relevant blogs to the user

Core Concepts

- Content Similarity:
Recommend blogs with similar topics, keywords, or meaning
 - Category Matching:
Prioritize blogs from the same or related categories
-

API Requirement

Endpoint

POST /ai/recommendations

Input

```
{
  "blogId": "123",
  "category": "frontend"
}
```

Output

```
{
  "recommendations": [
    {
      "blogId": "456",
      "title": "Getting Started with React"
    },
    {
      "blogId": "789",
      "title": "Beginner Guide to Web Development"
    }
  ]
}
```

Expected Outcome

- Relevant and meaningful recommendations
- No duplicate or same blog in results
- Fast response suitable for real-time UI usage

AI SEO Helper

Team Size: Sameera0103

Objective

Build an AI-powered SEO helper that generates SEO-optimized content for blogs, jobs, or service pages to improve search engine visibility.



Task Description

The system should generate:

- Meta Title (SEO-friendly, concise)
- Meta Description (engaging, keyword-focused)
- SEO Headings (H1, H2 suggestions)

Based on the given content or page information.



Core Concepts

- Keyword extraction
 - SEO best practices (length, clarity, intent)
 - Content relevance and readability
 - Optional: AI API (OpenAI / Gemini) for natural, marketing-focused text
-



API Requirement

Endpoint

POST /ai/seo-helper



Input

```
{  
  "title": "AI Blog Platform",  
  "content": "This blog explains how AI improves content creation and  
SEO."  
}
```



Output

```
{
```

```
"metaTitle": "AI Blog Platform for Smart Content & SEO",
"metaDescription": "Learn how an AI-powered blog platform helps
generate better content, SEO tags, and recommendations.",
"headings": [
  "How AI Improves Blog SEO",
  "Benefits of Using AI for Content Optimization",
  "AI Tools for Better Search Rankings"
]
```

✓ Expected Outcome

- SEO-optimized and human-readable output
- Proper length limits (Google-friendly)
- Consistent results with graceful fallback on failure

Blog Engagement Analyzer

Team Size: **RahulDewangan21**

Objective

Build a basic engagement analytics system to analyze how users interact with blog content and identify which blogs perform better.

Task Description

The system should track and analyze:

- Which blogs are read more frequently
- Time spent on a blog page
- Basic scroll depth (how much of the page is viewed)

This data will help understand user interest and content quality.

Core Concepts

- Page view tracking
 - Time-on-page calculation
 - Scroll depth measurement (basic percentage or milestones)
 - Aggregation of engagement metrics
-

API Requirement

Endpoint

POST /ai/blog-engagement

Input

```
{  
  "blogId": "123",  
  "timeOnPage": 95,  
  "scrollDepth": 70  
}
```

Output

```
{  
  "success": true  
}
```

Expected Insights

- Identify high-performing blogs
- Detect blogs with low engagement
- Support future recommendations and SEO decisions

Blog Content Quality Checker and Blog SEO Opportunity Finder

Team Size: **kbrakesh26 (both task) in weak**

Objective

Build a system to identify low-quality blog content, such as:

- Very short or spammy blogs
 - Duplicate or near-duplicate content (basic level)
-

Task Description

The system should:

1. Analyze blog content length and structure
 2. Flag blogs that are:
 - Too short to be meaningful
 - Repetitive or spam-like
 3. Detect basic duplicate content across blogs
-

Core Concepts

- Word count & minimum length threshold
 - Repetition ratio (same words repeated)
 - Basic text similarity (TF-IDF / cosine similarity)
-

API Requirement

Endpoint

POST /ai/content-quality

 Input

```
{  
  "blogId": "123",
```

```
"content": "Blog text here"
}
```

Output

```
{
  "isLowQuality": true,
  "isDuplicate": false,
  "reason": "Content too short"
}
```

Expected Outcome

- Low-quality blogs flagged automatically
 - Duplicate content detected at a basic level
 - Clear reason provided for each flag
-
-

Blog SEO Opportunity Finder

Objective

Help improve blog SEO by identifying missed SEO opportunities, such as:

- Long-tail keyword ideas
 - Internal linking suggestions
-

Task Description

The system should:

1. Analyze blog content and existing keywords
2. Suggest long-tail keywords related to the topic
3. Recommend internal links to other relevant blogs

Core Concepts

- Keyword extraction
 - Long-tail keyword expansion
 - Content similarity for internal linking
-

API Requirement

Endpoint

POST /ai/seo-opportunities

Input

```
{
  "blogId": "123",
  "content": "Blog content here"
}
```

Output

```
{
  "longTailKeywords": [
    "how to learn react for beginners",
    "react frontend best practices"
  ],
  "internalLinks": [
    {
      "blogId": "456",
      "title": "React Basics Guide"
    }
  ]
}
```

Expected Outcome

- Actionable SEO suggestions
- Better internal blog linking
- Improved organic search potential

Feasibility Note

- ✓ Both tasks are rule-based + lightweight AI friendly
- ✓ Achievable by 1 student in 1 week
- ✓ **High impact with low complexity**

Flow :

DETAILED PRODUCTION FLOW (REAL SYSTEM)

 Example: Blog Summary AI

[Frontend]

Admin clicks "Generate Summary"

|

v

[Backend Controller]

POST /api/ai/blog-summary

|

v

[Validation Layer]

- Blog exists?
- Content length ok?
- User allowed?

|

v

[Cache / DB Check]

Is summary already present?

|

+----+----+

|

YES

|

NO

|

|

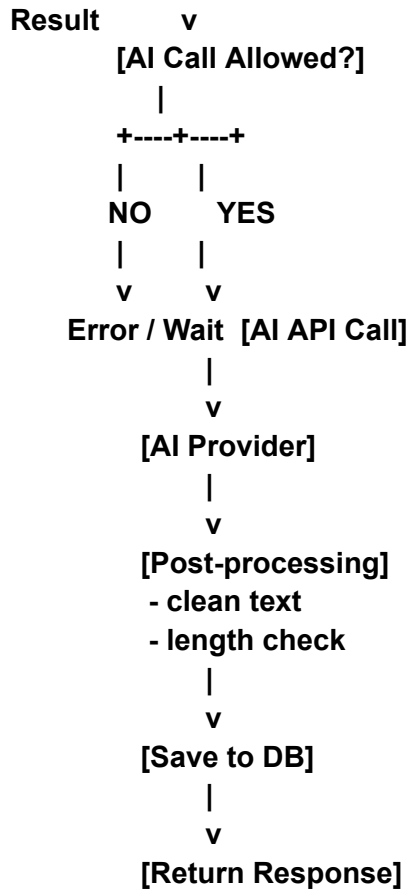
v

v

[Return] [Rate Limit Check]

Cached

|



Feature	AI API Used	ML / Algorithm Used	Reason / Why
Blog Summary (TL;DR)	✓ Yes	✗ No	Requires creative, natural language
SEO Title / Meta Description	✓ Yes	✗ No	Marketing tone + SEO optimization
Blog Tags Generator	✗ No	✓ Yes	Cheap, fast, and accurate
Semantic Search	✗ No	✓ Yes	Better speed and cost control
Content Recommendations	✗ No	✓ Yes	Deterministic and predictable
Related Blogs	✗ No	✓ Yes	Similarity-based logic
Content Quality Check	✗ No	✓ Yes	Rule-based evaluation

Spam Detection (Basic)	 No	 Yes	Heuristic-based detection
SEO Keyword Ideas	 Optional	 Mostly	Hybrid (rules + optional AI)
Internal Linking Suggestions	 No	 Yes	Graph & similarity logic

Note (Important)

For Blog Summary (TL;DR) and SEO Title / Meta Description, the primary approach is to use an AI API because these features require creative, natural, and marketing-friendly language.

However, if a rule-based or algorithmic approach does not provide acceptable accuracy or quality, the system must fall back to using an AI API to ensure:

- Better language fluency
- Clear intent representation
- Higher user satisfaction

This fallback decision should be made based on output quality and consistency, not just cost.